Estructuras de Datos y Algoritmos

Tema 9: Árboles y árboles binarios

Dr. Jorge Armando Ramírez Díaz

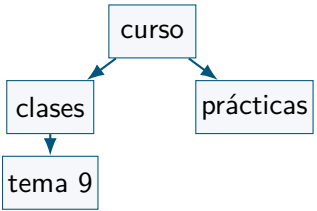
Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

De listas a árboles

Una lista organiza una **secuencia**. Un árbol organiza una **jerarquía**.



Avanzamos a un siguiente elemento.



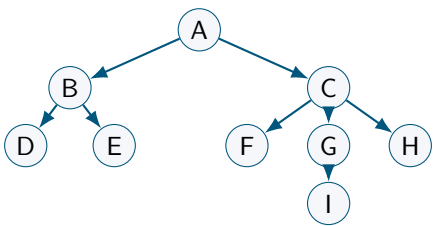
Elegimos entre varias ramas.

Ya utilizamos raíces, ramas y hojas al analizar árboles de recursión. Ahora representaremos esos vínculos como datos.

Árbol

Definición: árbol

Un árbol es una estructura jerárquica finita de nodos conectados mediante enlaces padre–hijo. Tiene un nodo llamado raíz, que no tiene padre. Cada otro nodo tiene exactamente un padre y se alcanza desde la raíz mediante un único camino.



Un solo nodo también es un árbol.

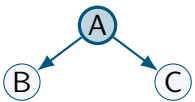


Un **árbol vacío**, es aquel que no tiene nodos ni raíz.

Relaciones

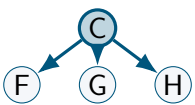
Raíz

Nodo sin padre desde el que se accede al árbol. Aquí es A.



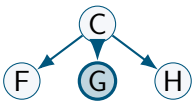
Padre

Nodo que tiene al menos un hijo. C es padre de F, G y H.



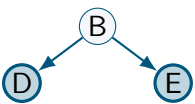
Hijo

Nodo conectado directamente a su padre. G es hijo de C.



Hermanos

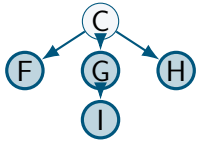
Nodos que tienen el mismo padre. D y E son hermanos.



Nodos

Descendientes

Nodos alcanzables por uno o más enlaces padre–hijo. De C: F, G, H e I.



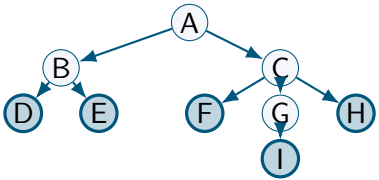
Ancestros

Nodos anteriores en el camino desde la raíz. De I: A, C y G.



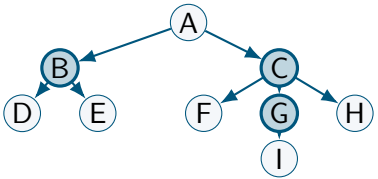
Hoja

Nodo sin hijos: D, E, F, H e I.



Nodo interno

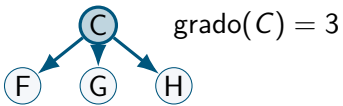
Nodo con hijos, excepto la raíz: B, C y G.



Grado y caminos

Grado de un nodo

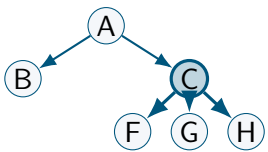
Número de hijos inmediatos. C tiene tres hijos.



$\text{grado}(C) = 3$

Grado del árbol

Mayor grado entre sus nodos. Aquí es 3: ningún nodo tiene más hijos que C.



Rama

Enlace directo entre un padre y uno de sus hijos. Por ejemplo, C a G.



Camino

Secuencia de nodos unidos por enlaces padre–hijo. La longitud cuenta enlaces, no nodos.

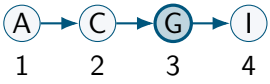


longitud = 2 enlaces

Medidas y subárboles

Nivel de un nodo

Número de enlaces desde la raíz más uno. La raíz ocupa el nivel 1.



Profundidad

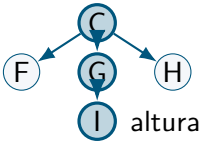
Número de enlaces desde la raíz hasta el nodo.



profundidad de G = 2

Altura de un nodo

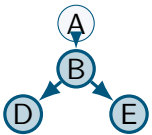
Máxima longitud del camino desde el nodo hasta una hoja.



altura de C = 2

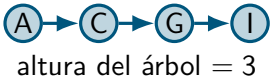
Subárbol

Un nodo y todos sus descendientes. Con raíz B: B, D y E.



Altura del árbol

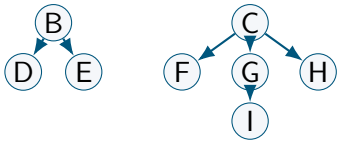
Altura de su raíz. El camino más largo desde A tiene tres enlaces.



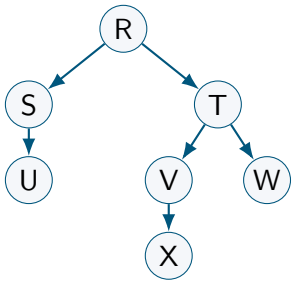
altura del árbol = 3

Bosque

Colección de árboles disjuntos. Al retirar A quedan dos árboles.



Ejemplo



Identifica:

1. Raíz, hojas y nodos internos.
2. Hermanos de V y ancestros de X.
3. Camino de R a X y su longitud.
4. Subárboles con raíces S y T.
5. Profundidad de V, altura de V y altura del árbol.

Respuesta

Raíz: R.

Hojas: U, W, X.

Internos: S, T, V.

Hermano de V: W.

Ancestros de X: R, T, V.

Camino: R→T→V→X; longitud 3.

Subárbol de S: S, U.

Subárbol de T: T, V, W, X.

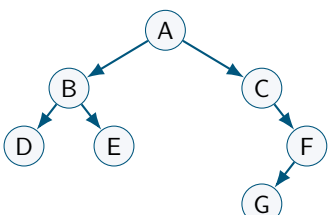
V: profundidad 2, altura 1.

Altura del árbol: 3.

Árbol binario

Definición: árbol binario

Un árbol binario es un árbol en el que cada nodo tiene como máximo dos hijos, distinguidos como hijo izquierdo e hijo derecho. Cualquiera de ellos puede estar ausente.

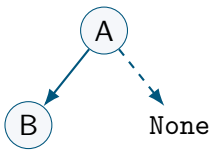


Cada subárbol es también un árbol binario.

La definición es **recursiva**; esto permitirá diseñar algoritmos con un caso base y llamadas sobre los subárboles.

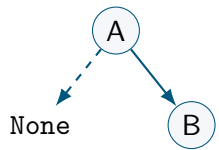
Izquierdo y derecho

Solo hijo izquierdo



El subárbol derecho es vacío.

Solo hijo derecho

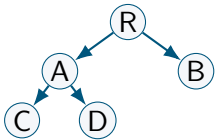


El subárbol izquierdo es vacío.

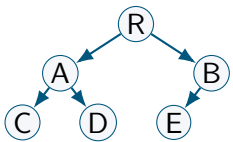
Los datos son los mismos, pero las **posiciones son distintas**. `None` señala ausencia de hijo; no representa otro nodo.

Formas de un árbol

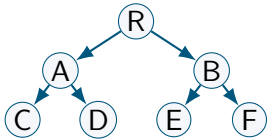
Estricto: cada nodo tiene 0 o 2 hijos.



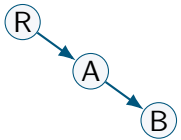
Completo: todos los niveles anteriores al último están llenos; el último se ocupa de izquierda a derecha.



Perfecto: todos los niveles están llenos.



Degenerado: los nodos forman una cadena.



Estas propiedades describen la **forma**, no el orden de los datos.

Nodos por nivel

Resultado: capacidad de un árbol binario

En el nivel $i \geq 1$ hay como máximo 2^{i-1} nodos. Un árbol binario no vacío de altura h tiene como máximo

$$n \leq 2^{h+1} - 1.$$

Nivel	Máximo	Acumulado
1	1	1
2	2	3
3	4	7
4	8	15

La raíz está en nivel 1 y profundidad 0.

Con altura 3 hay **cuatro niveles**, y a lo sumo 15 nodos. Un árbol perfecto alcanza el máximo.

Justificación

Sea N_i el número de nodos en el nivel i, comenzando en 1.

Base: $N_1 = 1 = 2^0$, porque solo hay una raíz.

Paso inductivo: si $N_i \leq 2^{i-1}$, cada nodo tiene a lo sumo dos hijos:

$$N_{i+1} \leq 2N_i \leq 2 \cdot 2^{i-1} = 2^i.$$

Un árbol de altura h tiene niveles desde 1 hasta $h+1$. Sumando:

$$n = \sum_{i=1}^{h+1} N_i \leq \sum_{i=1}^{h+1} 2^{i-1},$$

$$\sum_{i=1}^{h+1} 2^{i-1} = 1 + 2 + \cdots + 2^h = \frac{2^{h+1} - 1}{2 - 1} = 2^{h+1} - 1.$$

Hay igualdad cuando todos esos niveles están llenos.

Altura y tamaño

De $n \leq 2^{h+1} - 1$ obtenemos

$$n + 1 \leq 2^{h+1} \quad \Longrightarrow \quad \log_2(n+1) \leq h + 1.$$

Como h es entero, y un camino no puede contener más de n nodos:

Resultado: límites de la altura

Para un árbol binario con $n \geq 1$ nodos,

$$\lceil \log_2(n+1) \rceil - 1 \leq h \leq n - 1.$$

Con 7 nodos: un árbol perfecto tiene altura 2; una cadena, altura 6.

Ser binario **no garantiza** altura logarítmica.

TDA árbol

El TDA describe **qué podemos consultar o hacer**; la representación define cómo se guardan los nodos y sus vínculos.

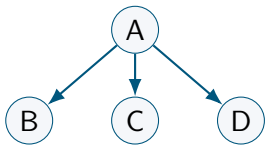
Operación	Comportamiento	Resultado
Consultar	Obtener raíz, hijos o dato.	Referencia o dato
Recorrer	Procesar todos los nodos en un orden.	Lista de datos
`contar_nodos(raiz)`	Contar nodos alcanzables.	Entero
`contar_hojas(raiz)`	Contar nodos sin hijos.	Entero
`altura(raiz)`	Medir el camino más largo hasta una hoja.	Entero

Construir un árbol requiere indicar **dónde y cómo se conectan** los nodos.

Árbol general en Python

```
class NodoGeneral:
    def __init__(self, dato):
        self.dato = dato
        self.hijos = []

a = NodoGeneral("A")
b = NodoGeneral("B")
c = NodoGeneral("C")
d = NodoGeneral("D")
a.hijos = [b, c, d]
```



`a.hijos` guarda referencias a B, C y D.

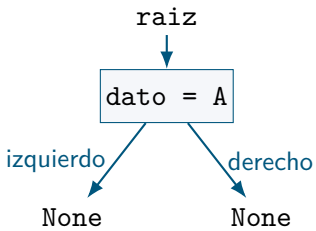
Cada constructor crea una **lista nueva**: los nodos no comparten accidentalmente su contenedor de hijos.

La lista de hijos conserva aquí su orden de izquierda a derecha.

Nodo binario en Python

```
class NodoBinario:
    def __init__(self, dato):
        self.dato = dato
        self.izquierdo = None
        self.derecho = None

raiz = NodoBinario("A")
```

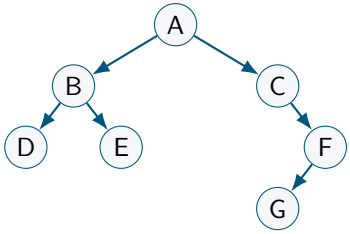


`raiz` refiere al nodo. Los dos atributos son referencias a raíces de subárboles o `None`.

Construir un árbol

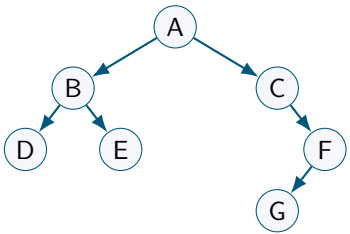
```
a = NodoBinario("A")
b = NodoBinario("B")
c = NodoBinario("C")
d = NodoBinario("D")
e = NodoBinario("E")
f = NodoBinario("F")
g = NodoBinario("G")
a.izquierdo, a.derecho = b, c
b.izquierdo, b.derecho = d, e
c.derecho = f
f.izquierdo = g
raiz = a
```

Los hijos que no se asignaron siguen siendo `None`.



7 nodos, altura 3.
Hojas: D, E, G.

Representación válida

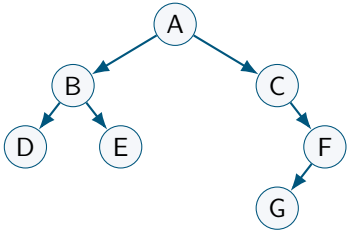


- Árbol vacío: `raiz is None`.
- Cada hijo es `None` o una referencia a un nodo.
- No hay ciclos en los enlaces de hijos.
- Ningún nodo alcanzable tiene dos padres.

Los subárboles izquierdo y derecho no comparten nodos.

Estas son **condiciones de representación**: deben conservarse al modificar la estructura.

Recuperación



Este árbol tiene **siete nodos**, altura 3 y hojas `D`, `E` y `G`.

En una lista simple avanzábamos mediante una única referencia `siguiente`.

Aquí puede haber dos subárboles pendientes.

¿En qué orden visitarías todos los nodos?

¿Cómo recordarías las ramas que todavía faltan?

Recorridos

Recorrer un árbol consiste en **visitar cada nodo una vez**, siguiendo un orden definido.

Visitar puede significar registrar el dato, consultarlo o realizar un cálculo.

En profundidad

Se explora un subárbol antes de continuar con otro.

El momento de visitar la raíz distingue:

- Preorden.
- Inorden.
- Postorden.

La recursión conserva las llamadas pendientes.

Por niveles

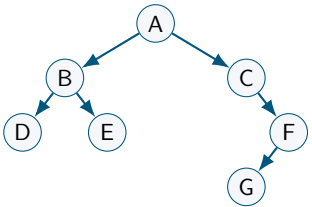
Se visitan primero los nodos del nivel 1, luego los del nivel 2, y así sucesivamente.

Dentro de cada nivel usaremos el orden de izquierda a derecha.

Una cola conserva los nodos pendientes en el orden adecuado.

Preorden

Orden: raíz, subárbol izquierdo, subárbol derecho.



Procedimiento Preorden(nodo)

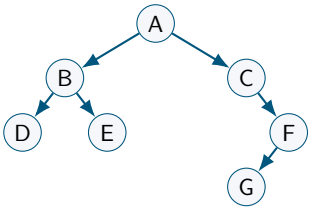
Si nodo es vacío:
    Retornar
Visitar(nodo)
Preorden(nodo.izquierdo)
Preorden(nodo.derecho)

La raíz de cada subárbol se procesa **antes** que sus descendientes.

Resultado:
`A, B, D, E, C, F, G.`

Inorden

Orden: subárbol izquierdo, raíz, subárbol derecho.



Procedimiento Inorden(nodo)

```
Si nodo es vacío:
    Retornar
Inorden(nodo.izquierdo)
Visitar(nodo)
Inorden(nodo.derecho)
```

La raíz se procesa **entre** sus dos subárboles.

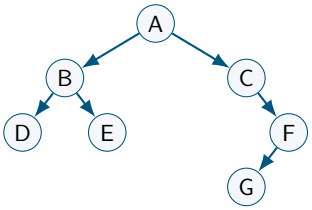
Resultado:

`D, B, E, A, C, G, F.`

En un árbol binario cualquiera, este recorrido **no garantiza** datos ordenados por valor.

Postorden

Orden: subárbol izquierdo, subárbol derecho, raíz.



Procedimiento Postorden(nodo)

Si nodo es vacío:
    Retornar
Postorden(nodo.izquierdo)
Postorden(nodo.derecho)
Visitar(nodo)

La raíz se procesa **después** de sus descendientes.

Resultado:

`D, E, B, G, F, C, A.`

Es útil cuando el cálculo de un nodo necesita primero los resultados de sus hijos.

Elegir un recorrido

El objetivo y las dependencias determinan cuándo procesar cada nodo.

Recorrido	Momento	Ejemplos de uso
Preorden	Antes de sus descendientes.	Mostrar una jerarquía desde sus superiores; generar notación prefija.
Inorden	Entre los subárboles izquierdo y derecho.	Obtener valores ordenados en un árbol binario de búsqueda; notación infija con paréntesis.
Postorden	Después de sus descendientes.	Calcular altura, combinar conteos de subárboles y evaluar expresiones.

El procesamiento postorden puede combinar resultados directamente, sin construir una lista del recorrido.

Con una visita de costo constante, los tres recorridos completos cuestan $\Theta(n)$. Cambia el **orden de procesamiento**, no la cantidad de nodos visitados.

Recorrido en Python

Visitar significa realizar una operación sobre el nodo. En este ejemplo usamos `print` para mostrar su dato.

```
def preorden(nodo):
    if nodo is None:
        return

    print(nodo.dato)
    preorden(nodo.izquierdo)
    preorden(nodo.derecho)
```

Otras formas de visitar:

- Guardar el dato en una lista.
- Acumular su valor en una suma.
- Comprobar una propiedad.
- Aplicar una operación al dato.

`preorden(None)` no imprime datos.

Para cambiar el recorrido, mueve el procesamiento del nodo: **entre** las dos llamadas recursivas para inorden; **después** de ambas para postorden.

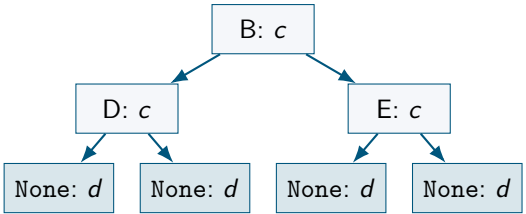
Árbol de recursión

Asignamos $c > 0$ al caso con nodo y $d > 0$ al caso `None`; suponemos una visita de costo constante (modelo RAM).

$$T(0) = d, \qquad T(n) = T(n_{\text{izq}}) + T(n_{\text{der}}) + c, \quad n \geq 1.$$

Los tamaños cumplen $n_{\text{izq}} + n_{\text{der}} = n - 1$, sin tener que ser iguales.

Ejemplo: subárbol de B ($n = 3$).



Trabajo en el ejemplo

Llamadas	Cantidad	Costo
B	1	c
D y E	2	$2c$
`None`	4	$4d$

$$T(3) = c + 2c + 4d.$$

Costo total

Sumamos n costos c de llamadas con nodos y $n+1$ costos d de llamadas a `None`:

$$\begin{aligned} T(n) &= nc + (n+1)d \\ &= nc + nd + d \\ &= (c+d)n + d. \end{aligned}$$

Cota inferior. Como $d > 0$, añadirlo no disminuye el costo:

$$T(n) = (c+d)n + d \geq (c+d)n.$$

Cota superior. Para $n \geq 1$, se cumple $d \leq dn$:

$$\begin{aligned} T(n) &= (c+d)n + d \\ &\leq (c+d)n + dn = (c+2d)n. \end{aligned}$$

Ambas cotas son múltiplos de n con constantes positivas. Por tanto,

$$\boxed{T(n) \in \Theta(n).}$$

Por niveles

Procedimiento PorNivel(raiz)

```
Si raíz es vacía:
    Retornar
Q ← cola vacía
Encolar(Q, raiz)
Mientras Q no esté vacía:
    nodo ← Desencolar(Q)
    Visitar(nodo)
    Si nodo.izquierdo no es vacío:
        Encolar(Q, nodo.izquierdo)
    Si nodo.derecho no es vacío:
        Encolar(Q, nodo.derecho)
```

Pendientes: una cola de referencias a nodos.

1. Encolar la raíz, si existe.
2. Retirar el nodo del frente y procesarlo.
3. Encolar su hijo izquierdo y después su hijo derecho, si existen.
4. Repetir mientras la cola no esté vacía.

FIFO permite procesar los nodos de un nivel antes que sus descendientes.

Resultado: `A, B, C, D, E, F, G.`

Algoritmo con cola

Usamos `from collections import deque`.

```
def por_nivel(raiz):
    if raiz is None:
        return []
    pendientes = deque([raiz])
    resultado = []
    while pendientes:
        nodo = pendientes.popleft()
        resultado.append(nodo.dato)
        if nodo.izquierdo is not None:
            pendientes.append(nodo.izquierdo)
        if nodo.derecho is not None:
            pendientes.append(nodo.derecho)
    return resultado
```

La cola conserva **nodos**; la lista de salida conserva sus **datos**.

Las referencias `None` no se encolan.

En un árbol válido, cada nodo se retira una vez: n iteraciones con trabajo constante amortizado. Por tanto, el costo es $\Theta(n)$.

Contar nodos

Sea $N(T)$ el número de nodos del árbol T.

$$N(\varnothing) = 0, \qquad N(T) = 1 + N(T_{\text{izq}}) + N(T_{\text{der}}) \quad \text{si } T \neq \varnothing.$$

Función ContarNodos(raiz)

Si raíz es vacía:
    Retornar 0
$n_{\text{izq}} \leftarrow$ ContarNodos(raiz.izquierdo)
$n_{\text{der}} \leftarrow$ ContarNodos(raiz.derecho)
Retornar $1 + n_{\text{izq}} + n_{\text{der}}$

El 1 cuenta la raíz de este subárbol.

Los otros términos cuentan todos los nodos de cada lado.

Contar nodos

```
def contar_nodos(raiz):
    if raiz is None:
        return 0
    n_izq = contar_nodos(raiz.izquierdo)
    n_der = contar_nodos(raiz.derecho)
    return 1 + n_izq + n_der
```

El árbol de llamadas es el mismo que en preorden.

Aquí se combinan los resultados **al regresar** de ambos subárboles: procesamiento postorden.

Costo. Con costos locales constantes $c, d > 0$ propios de esta función, y $n_{\text{izq}} + n_{\text{der}} = n - 1$ para $n \geq 1$, recuperamos la recurrencia de preorden:

$$T(0) = d, \qquad T(n) = T(n_{\text{izq}}) + T(n_{\text{der}}) + c \quad \Longrightarrow \quad \boxed{T(n) \in \Theta(n).}$$

Contar hojas

Sea $L(T)$ el número de hojas del árbol T.

$$L(T) = \begin{cases} 0, & T = \varnothing, \\ 1, & T \text{ tiene un único nodo}, \\ L(T_{\text{izq}}) + L(T_{\text{der}}), & \text{en otro caso.} \end{cases}$$

Función ContarHojas(raiz)

Si raíz es vacía:
    Retornar 0

Si ambos hijos de raíz son vacíos:
    Retornar 1

hojas_izq ← ContarHojas(raiz.izquierdo)
hojas_der ← ContarHojas(raiz.derecho)
Retornar hojas_izq + hojas_der

Contar hojas

```
def contar_hojas(raiz):
    if raiz is None:
        return 0
    if (raiz.izquierdo is None
            and raiz.derecho is None):
        return 1
    hojas_izq = contar_hojas(raiz.izquierdo)
    hojas_der = contar_hojas(raiz.derecho)
    return hojas_izq + hojas_der
```

Costo. Con n nodos y costos locales constantes $c, d, e > 0$:

$$T(n) = \begin{cases} d, & n = 0, \\ e, & n = 1, \\ T(n_{\text{izq}}) + T(n_{\text{der}}) + c, & n \geq 2 \end{cases} \quad \Longrightarrow \quad \boxed{T(n) \in \Theta(n).}$$

Para $n \geq 2$, $n_{\text{izq}} + n_{\text{der}} = n - 1$.

Calcular altura

Contamos **enlaces** en el camino descendente más largo. Para extender la definición al árbol vacío, fijamos:

$$H(\varnothing) = -1, \qquad H(T) = 1 + \max\{H(T_{\text{izq}}), H(T_{\text{der}})\} \quad \text{si } T \neq \varnothing.$$

Función Altura(raiz)

Si raíz es vacía:
    Retornar -1
izquierda $\leftarrow$ Altura(raiz.izquierdo)
derecha $\leftarrow$ Altura(raiz.derecho)
Retornar $1 + \max\{\text{izquierda}, \text{derecha}\}$

Si la raíz es una hoja:

$$1 + \max\{-1, -1\} = 0.$$

El -1 es una convención para el caso vacío; no representa un camino de longitud negativa.

Calcular altura

```
def altura(raiz):
    if raiz is None:
        return -1
    izquierda = altura(raiz.izquierdo)
    derecha = altura(raiz.derecho)
    return 1 + max(izquierda, derecha)
```

Se recorren **ambos subárboles** antes de elegir la mayor altura.

Comparar los dos resultados con `max` cuesta $\Theta(1)$.

Costo. Con costos locales constantes $c, d > 0$ propios de esta función, y $n_{\text{izq}} + n_{\text{der}} = n - 1$ para $n \geq 1$, recuperamos la recurrencia de preorden:

$$T(0) = d, \qquad T(n) = T(n_{\text{izq}}) + T(n_{\text{der}}) + c \quad \Longrightarrow \quad \boxed{T(n) \in \Theta(n).}$$

El costo depende del número de nodos, no solamente de la altura.

Costos

Para un árbol no vacío de n nodos:

Algoritmo	Costo temporal
Preorden, inorden y postorden recursivos	$\Theta(n)$
Recorrido por niveles	$\Theta(n)$
Contar nodos, contar hojas y altura	$\Theta(n)$

Cada nodo se procesa una vez, con trabajo local constante. En las rutinas recursivas, las llamadas a subárboles vacíos añaden a lo sumo trabajo lineal.

Suponemos visitas y operaciones elementales de costo constante.

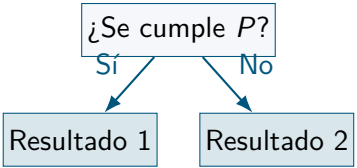
Árbol de decisión

Definición: árbol de decisión

Un árbol que representa una secuencia de preguntas: los nodos con hijos contienen preguntas, las ramas indican sus respuestas y las hojas contienen los resultados.

Decisiones binarias

- Cada pregunta admite dos respuestas: sí o no.
- Según la respuesta, se elige uno de los dos hijos.
- Al alcanzar una hoja, se obtiene un resultado.

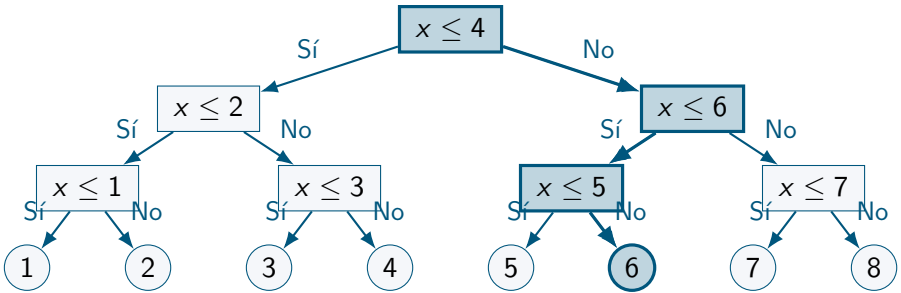


El árbol determina **qué preguntar después**; no necesitamos visitar todas sus ramas para obtener una respuesta.

Referencia: A Course in Machine Learning, cap. 1, sección 1.3. Aquí no estudiaremos entrenamiento.

Ejemplo: identificar un número

Una persona conoce $x \in \{1, \ldots, 8\}$. Solo podemos hacerle preguntas de **sí o no** para identificarlo.



Para $x = 6$: $x \leq 4$: **no**; $x \leq 6$: **sí**; $x \leq 5$: **no**. Se alcanza la hoja 6 tras tres preguntas.

Algoritmo

Precondición: árbol finito y no vacío; cada pregunta tiene un hijo para sí y otro para no, y las respuestas son correctas.

Función Identificar(raiz)

```
nodo ← raiz
Mientras nodo tenga hijos:
    respuesta ← Preguntar(nodo.dato)
    Si respuesta es sí:
        nodo ← nodo.izquierdo
    Si no:
        nodo ← nodo.derecho
Retornar nodo.dato
```

Preguntar obtiene la respuesta de quien conoce el número.

Convención: sí corresponde al hijo izquierdo; no, al derecho.

En cada paso, el número buscado sigue entre los resultados posibles del subárbol elegido.

Terminación: cada paso desciende un enlace; el árbol es finito. La hoja alcanzada contiene el resultado.

Juegos

Definición: teoría de juegos

Estudia situaciones en las que el resultado de cada participante depende tanto de sus propias decisiones como de las decisiones de otros.

Elementos de un juego

- **Jugadores:** quienes toman decisiones.
- **Acciones:** elecciones permitidas.
- **Resultados y pagos:** qué ocurre al terminar y cómo lo valora cada jugador.

Nos limitaremos a juegos

- Finitos y de dos jugadores.
- Por turnos y sin azar.
- Con información perfecta: se conocen las jugadas anteriores.
- De suma cero: los pagos de ambos jugadores suman cero.

Ahora una rama representa una **acción elegida por un jugador**, no una respuesta a una pregunta.

Las fichas

- Inicialmente hay **cuatro fichas**. Comienza el jugador A.
- A y B alternan turnos.
- En cada turno se retiran una o dos fichas, sin exceder las disponibles.
- Gana quien retira la última ficha.

Escribimos un estado como (r, J): fichas restantes y jugador que movería.

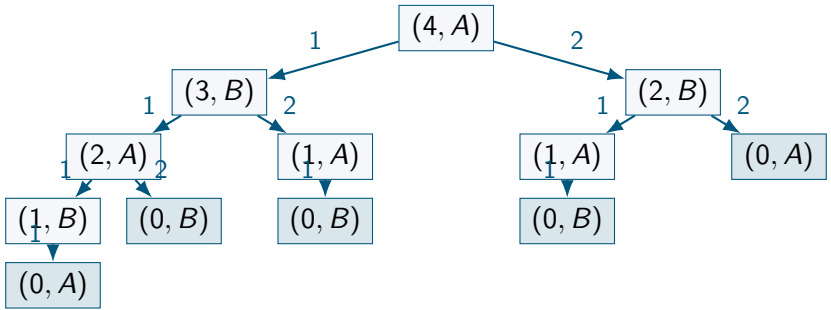


En esta partida gana A. En $(0, B)$, B sería el siguiente jugador, pero la partida ya terminó.

Una partida favorable **no demuestra todavía** que podamos ganar ante cualquier respuesta del adversario.

Árbol de juego

La raíz es el estado inicial; cada rama indica cuántas fichas se retiran. Los nodos con $r = 0$ son terminales.



Hay 12 nodos y 5 hojas. Un mismo estado puede aparecer en nodos distintos: cada camino representa una **historia de jugadas diferente**. No compartimos nodos entre padres.

Estrategias y pagos

Definición: estrategia

Un plan que indica qué acción elegir en cada situación en la que corresponde jugar, incluso si esa situación no aparece en la partida realizada.

Una estrategia para A

Estado	Retirar
$(4, A)$	1
$(2, A)$	2
$(1, A)$	1

Si B retira una ficha desde $(3, B)$, A retira dos; si B retira dos, A retira una.

Pago terminal desde la perspectiva de A

$$u_A(0, A) = -1, \qquad u_A(0, B) = +1.$$

- En $(0, A)$, B retiró la última ficha.
- En $(0, B)$, A retiró la última ficha.

El pago de B es $u_B = -u_A$.

Los valores **siempre** expresan el resultado para A, también cuando decide B.